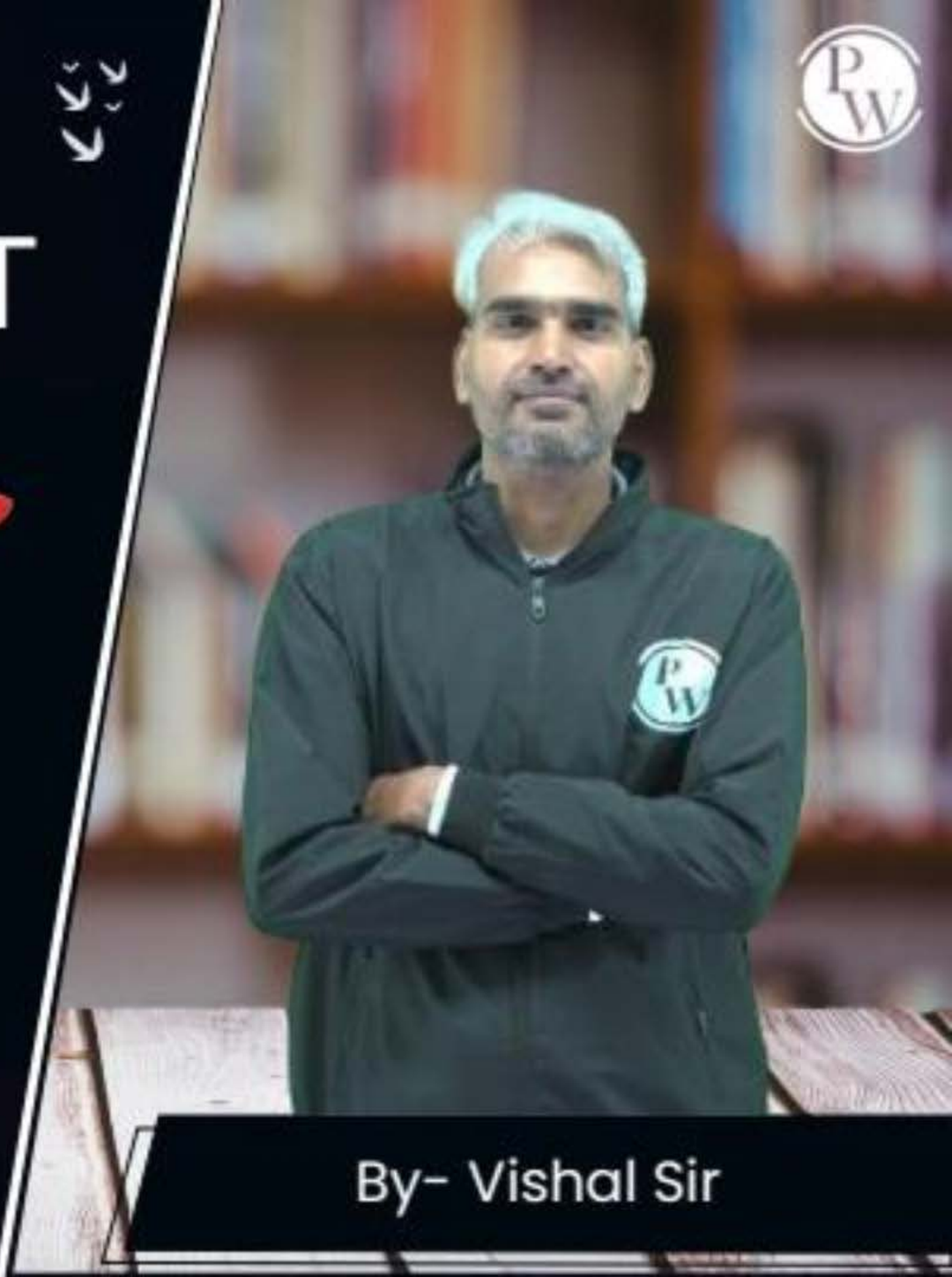
Computer Science & IT



Compiler Design

Intermediate Code Generation
&
Code Optimization

Lecture No. 01



By- Vishal Sir

Recap of Previous Lecture



Topic Difference between SDD and SDT

Topic Syntax-Directed Translation Schemes for S-attributed SDD

Topic Syntax-Directed Translation Schemes for L-attributed SDD

Slide

Topics to be Covered



Topic Intermediate Representation

Topic Code optimization Techniques

Slide



Topic : Why intermediate code

Portability of Compiler increases



L.A
↓
Syn A
↓
Sem A

Analysis Phase

{ Source Code written in High level language is analyzed }

Intermediate Representation & Machine Independent optimization

Code Generation
↓
Machine dependent Code optimization

Synthesis Phase

Will be same for all Machines

Analysis Phase

Intermediate Rep

Different for different Machines

Code generator for M/C 1

Code generator for M/C 2

Code generator for M/C - 3

Machine 1

Machine 2

Machine 3

Slide



Topic : Intermediate Representation

- Linear Form
  1. Postfix
  2. Three Address Code
  3. Static Single Assignment form (SSA form)
- Non-linear Form
  - Abstract Syntax Tree (Syntax Tree)
  - Directed Acyclic Graph (DAG)
  - Control Flow graph

Study After Code optimization

Slide

Consider following Expression

$$x = (a+b) * (a+b+c)$$

t_1 : $(a+b)$

t_2 : $(a+b)$

t_3 : $(a+b+c)$

t_4 : $t_1 * t_3$

Three Address Code — A representation with at most three addresses in each statement

$$\begin{cases} t_1 = a + b \\ t_2 = a + b \\ t_3 = t_2 + c \\ t_4 = t_1 * t_3 \\ x = t_4 \end{cases}$$

Consider following Expression

$x = (a+b) * (a+b+c)$

Post-fix

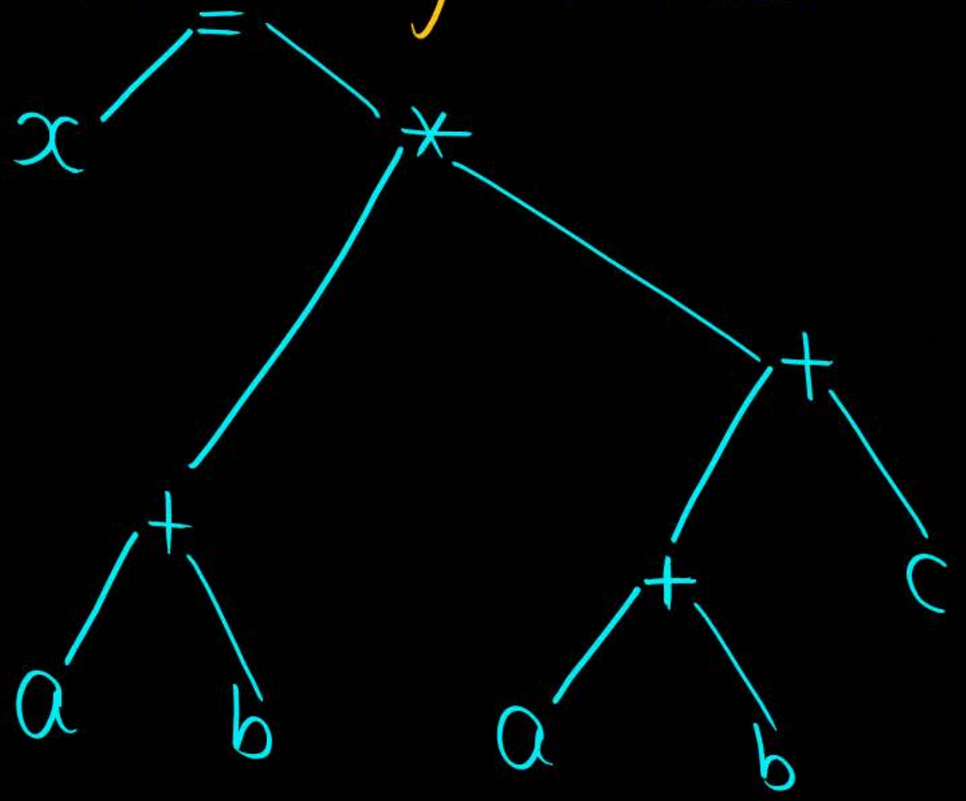
ab+ ab+c+ *

in-fix	Post fix	Prefix
op_1 op^h op_2	op_1 op_2 op^h	op^h op_1 op_2

Consider following Expression

$x = (a+b) * (a+b+c)$

Abstract Syntax Tree

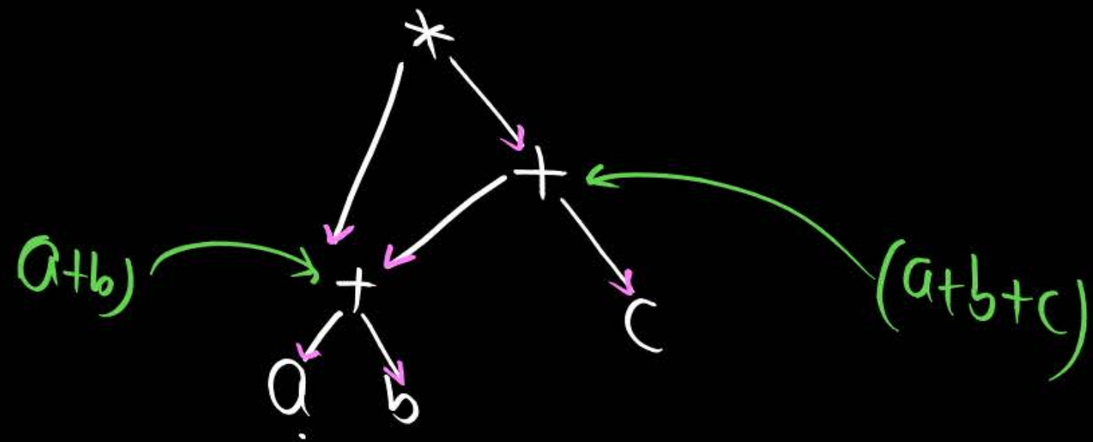


Consider following Expression

$x = (a+b) * (a+b+c)$

Directed Acyclic Graph (DAG)

Main Concept is to eliminate Common Sub-expression



✡ Three Address Code :- In 3-add^{r} Code representation

each statement can contain at most three addresses

* Types of three add^{r} Code :-

① $x = y \; op^{h} \; z$ (binary op^{h})

② $x = op^{h} \; y$ (Unary op^{h})

③ $x = y$

④ if x relational operator y then goto L

⑤ goto L

add^{r} w.r.t destination operand

Addresses w.r.t source operands

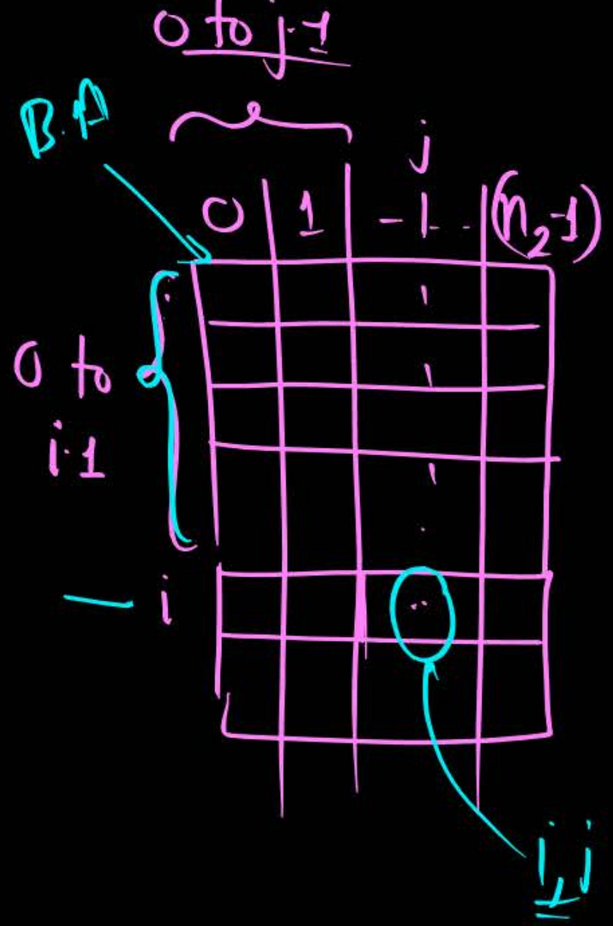
⑥ $A[i] = x$ // $*(A+i) = x$

$y = B[j]$

Add^r w.r.t Base add^r of array A

add^r w.r.t offset value

add^r



- Note:- $A[i,j] = x$ is not a three add^r code

Let Array A is defined as $A[n_1][n_2]$

$A[i,j] = ? = B.A + ((i * n_2) + j) * w$

* Representation of three address Code :-

- Quadruples
- Triples
- Indirect Triples

Consider the following Expression

t_2 t_3

$-(a+b) * (c+d) + (a+b+c)$

t_4 t_6

Three Add^{r} Code

(i) $t_1 = a+b$

(ii) $t_2 = -t_1$

(iii) $t_3 = c+d$

(iv) $t_4 = t_2 * t_3$

(v) $t_5 = a+b$

(vi) $t_6 = t_5+c$

(vii) $t_7 = t_4+t_6$

Consider the following Expression

t_2 t_3 t_4 t_6

$$-(a+b) * (c+d) + (a+b+c)$$

Quadruples Notation { Result of the statement is stored }

Three Add^{r} Code

(i) $t_1 = a + b$
(ii) $t_2 = -t_1$
(iii) $t_3 = c + d$
(iv) $t_4 = t_2 * t_3$
(v) $t_5 = a + b$
(vi) $t_6 = t_5 + c$
(vii) $t_7 = t_4 + t_6$

	Operator	Operand1	Operand2	Result
(i)	+	a	b	t_1
(ii) Unary	−	t_1	−	t_2
(iii)	+	c	d	t_3
(iv)	*	t_2	t_3	t_4
(v)	+	a	b	t_5
(vi)	+	t_5	c	t_6
(vii)	+	t_4	t_6	t_7

Consider the following Expression

$-(a+b) * (c+d) + (a+b+c)$

t_2 : $-(a+b)$, t_3 : $(c+d)$, t_4 : $-(a+b) * (c+d)$, t_6 : $(a+b+c)$

Triple Notation { Result is not stored }

Three Add^{r} Code

(i) $t_1 = a+b$ (1)

(ii) $t_2 = -t_1$ (2)

(iii) $t_3 = c+d$ (3)

(iv) $t_4 = t_2 * t_3$ (4)

(v) $t_5 = a+b$ (5)

(vi) $t_6 = t_5 + c$ (6)

(vii) $t_7 = t_4 + t_6$ (7)

Add^{r}	Operator	Operand$_1$	Operand$_2$
	+	a	b
	−	(1)	
	+	c	d
	*	(2)	(3)
	+	a	b
	+	(5)	c
	+	(4)	(6)

Quadruples :-

Advantage :- Execution will take less time

Disadvantage :- More storage space is required

Triples :-

Advantage :- less storage space (because we don't need to store the result)

Disadvantage :- It takes more time, because every time we need the result of an statement we need to execute it.

Note :- In triple notation if we move the statements in the memory, then we will have to modify the addr fields in the representation

Consider the following Expression

$$\underbrace{-(a+b)}_{t_2} * \underbrace{(c+d)}_{t_3} + \underbrace{(a+b+c)}_{t_6}$$

t_4

Indirect Triple :-

In indirect triple notation we use an extra array for execution

Three Add^{r} Code

(i) $t_1 = a + b$ (35)
(ii) $t_2 = -t_1$ (40)
(iii) $t_3 = c + d$ (45)
(iv) $t_4 = t_2 * t_3$ (50)
(v) $t_5 = a + b$ (55)
(vi) $t_6 = t_5 + c$ (60)
(vii) $t_7 = t_4 + t_6$ (65)

Add^{r}	Operator	operand1	operand2
100	+	a	b
(500)	−		
	+	c	d
(501)	*		(502)
	+	a	b
(504)	+		c
(503)	+		(505)

	fixed Add^{r}	Actual Add^{r}
(i)	500	35 100
(ii)	501	40
(iii)	502	45
(iv)	503	50
(v)	504	55
(vi)	505	60
(vii)	506	65

* In Case of indirect triple two memory accesses will be required if we need output of any specific statement, i.e. It will take more time

Back-patching :- { In this concept initially we will leave the add^{r} field empty and it is filled at later stage }

eg: Write three-add^{r} code for following expression

* if (a < b) then t = 1 else e = 0

Add^{r}: i : if (a < b) then goto (i+3)

w.r.t else { i+1 : e = 0 ; i+2 : goto (i+4) }

when if Cond^{n} is true { i+3 : t = 1 }

Statement after if-else → i+4 :

add^{r} w.r.t. t = 1

Slide

* Write three add^{r} code for following expression..



if $(a < b)$ && $(c > d)$ then $t = 1$ else $e = 0$

```
i   : if (a < b) then goto (i+2)
i+1 : goto (i+3)
i+2 : if (c > d) then goto (i+5)
i+3 : e = 0
i+4 : goto (i+6)
i+5 : t = 1
i+6 :
```

add^{r} to check $(c > d)$

add^{r} w.r.t else statement

Statement w.r.t then Part of original expression

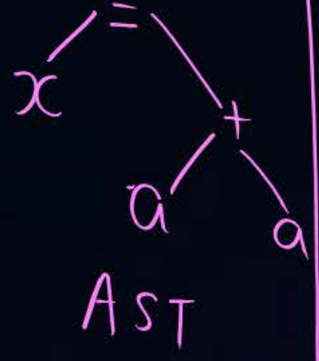
to jump the statement w.r.t. true Cond^{n}

* Directed Acyclic graph :-

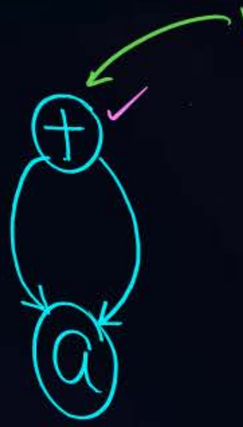
{ If you have already created a node for an expression, then use the same node whenever value of that expression is needed }

Construct DAG for following expressions

① $x = a + a$



AST



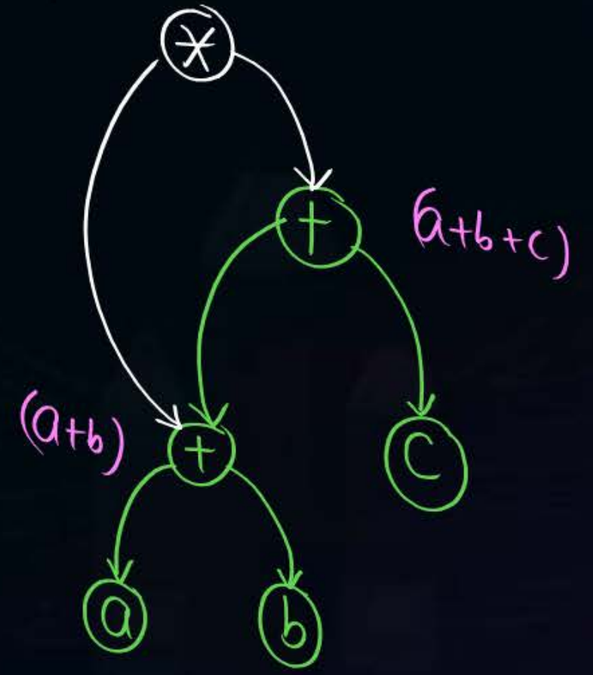
Value w.r.t x is available at this node

DAG

② $x = a + a + a$



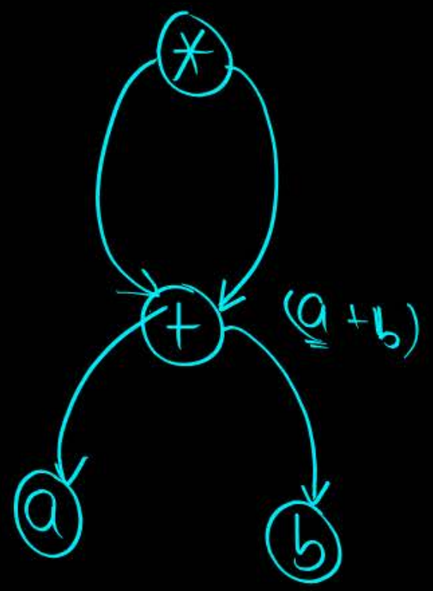
③ $x = (a+b) * (a+b+c)$



Slide

a * a

④ $y = (a+b) * (a+b)$

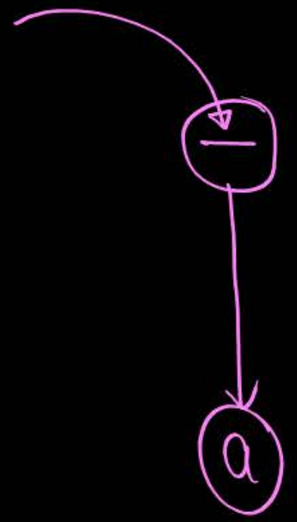


Types of questions Possible

① Given an expression identify the number of nodes and edges in the DAG.

② Given a DAG, identify the expression corresponding to that DAG

(5) $Z = -a$





2 mins Summary

Topic Types of Intermediate Representation

Topic Static Single Assignment

Topic Code Optimization Techniques

Slide

THANK - YOU